

Unit 2 – Assignment

Problem 3: Choosing Architecture for a Scalable Food Delivery Platform

Scenario

A startup is developing a food delivery platform similar to Swiggy or Zomato. The platform should be capable of handling thousands of customer orders simultaneously, especially during peak hours. The development team needs to choose the most suitable software architecture to ensure scalability, reliability, and efficient performance.

1. Recommended Architecture Choice

Recommended Architecture: Microservices Architecture

A **Microservices Architecture** is the most suitable choice for a large-scale food delivery platform because it divides the application into independent services. Each service performs a specific business function and can be developed, deployed, and maintained separately.

For example, services such as **Order Management**, **Payment**, and **Delivery Tracking** can operate independently while communicating through APIs.

2. Three Technical Reasons for Choosing Microservices

1. High Scalability

Each microservice can be scaled independently based on demand.

Example:

During lunch or dinner hours, the **Order Service** may receive thousands of requests. Instead of increasing the resources for the entire application, only the Order Service needs additional servers.

2. Better Fault Isolation

If one service experiences a failure, the remaining services continue to operate normally.

Example:

If the Payment Service becomes temporarily unavailable, customers can still browse restaurants, search menus, and add items to their cart.

3. Faster Development and Deployment

Different development teams can work on different services simultaneously without affecting each other's work.

Example:

One team can improve the Delivery Tracking feature while another team updates the Payment Service, enabling faster software releases.

3. Benefits of Event-Driven Architecture

An **Event-Driven Architecture (EDA)** allows different microservices to communicate through events instead of directly calling each other.

How It Works

When a customer places an order:

1. The Order Service creates the order.
2. An **"Order Created"** event is generated.
3. The Payment Service receives the event and processes payment.
4. After successful payment, a **"Payment Successful"** event is generated.
5. The Delivery Service receives the event and assigns a delivery partner.
6. The Notification Service informs both the customer and the restaurant.

Advantages

- Services communicate without tight dependency.
- Faster processing of customer requests.
- Easier to add new features in the future.
- Improves system reliability.
- Supports asynchronous communication during heavy traffic.

4. Suitable Microservices

1. Order Management Service

- Accepts customer orders.
- Stores order details.
- Updates order status.
- Cancels or modifies orders.

2. Payment Service

- Processes online payments.
- Verifies transactions.
- Handles refunds.
- Generates payment receipts.

3. Delivery Management Service

- Assigns delivery partners.
- Tracks delivery location.
- Calculates delivery routes.
- Updates delivery status.

Conclusion

A **Microservices Architecture** is the most appropriate solution for a scalable food delivery platform because it offers flexibility, independent deployment, high availability, and easier maintenance. Combining it with **Event-Driven Architecture** enables efficient communication between services, allowing the system to process thousands of customer orders quickly and reliably. This architecture supports business growth while providing a smooth and responsive experience for customers.

Simple Architecture Flow:

